

Reporte: Paradigma lógico con Prolog

Alumno: Fabricio Reyes Villavicencio

Materia: Paradigmas de la programación

Profesor: José Carlos Gallegos Mariscal

Grupo: 941

Fecha: 23/05/2028

Introducción

En la presente práctica se trabajó con el paradigma lógico mediante el lenguaje de programación Prolog. A diferencia de los paradigmas imperativo y funcional, la programación lógica no se enfoca en indicar paso a paso cómo resolver un problema, sino en describir hechos, reglas y relaciones lógicas para que el sistema encuentre automáticamente una solución.

Prolog es un lenguaje basado en el cálculo de predicados de primer orden y utiliza mecanismos como la unificación y el backtracking para resolver consultas. Los programas se construyen mediante una base de conocimientos compuesta por hechos y reglas que posteriormente son utilizadas por el motor lógico para realizar inferencias.

Durante esta práctica se estudiaron conceptos fundamentales del paradigma lógico, la sintaxis básica de Prolog y la resolución de problemas clásicos como el problema del mono y la banana y las Torres de Hanoi.

Primera sesión

Introducción a Prolog

Prolog es un lenguaje de programación lógica utilizado principalmente en inteligencia artificial, sistemas expertos y resolución de problemas mediante inferencia.

En Prolog, los programas se componen de:

- Hechos.
- Reglas.
- Consultas.

El sistema analiza estas definiciones para determinar si una consulta puede ser resuelta de manera lógica.

Entre las características principales del paradigma lógico destacan:

- Uso de inferencia automática.
- Programación declarativa.
- Representación del conocimiento mediante reglas.
- Backtracking automático.

- Uso de recursividad.

Instalación y verificación del entorno

Para trabajar con Prolog se utilizó SWI-Prolog, uno de los entornos más utilizados para este lenguaje.

La instalación puede realizarse desde la terminal mediante el siguiente comando:

```
winget install SWI-Prolog.SWI-Prolog
```

Posteriormente se verificó la instalación utilizando:

```
swipl --version
```

Para iniciar el intérprete interactivo se utilizó:

```
swipl
```

Los archivos de Prolog utilizan la extensión `.pl`.

Segunda sesión

Sintaxis de hechos

Los hechos representan afirmaciones verdaderas dentro de la base de conocimientos.

Ejemplo:

```
animal(gato).
```

Este hecho indica que gato pertenece a la categoría animal.

También pueden existir relaciones entre varios elementos:

```
le_gusta(juan, pizza).
```

La instrucción anterior representa que a Juan le gusta la pizza.

En Prolog:

- Los identificadores comienzan con minúscula.
- Las variables comienzan con mayúscula.
- Cada cláusula termina con punto.

Sintaxis de reglas

Las reglas permiten definir condiciones lógicas para inferir nueva información.

Estructura general:

```
conclusion :- condicion.
```

Ejemplo:

```
feliz(X) :- juega(X).
```

La regla anterior establece que una persona es feliz si juega.

También pueden utilizarse múltiples condiciones:

```
salir(X) :- termino_tarea(X), tiene_tiempo(X).
```

En este caso ambas condiciones deben cumplirse para que la conclusión sea verdadera.

Relaciones y consultas

Las relaciones son fundamentales en Prolog, ya que permiten conectar objetos mediante reglas lógicas.

Ejemplo:

```
padre(carlos, ana).  
madre(maria, ana).
```

A partir de estas relaciones pueden construirse reglas más complejas:

```
progenitor(X, Y) :- padre(X, Y).  
progenitor(X, Y) :- madre(X, Y).
```

Las consultas permiten preguntar información a la base de conocimientos:

```
?- progenitor(carlos, ana).
```

Si la relación es válida, Prolog responderá con **true**.

Tercera sesión

Problema del mono y la banana

Uno de los ejercicios desarrollados fue el problema del mono y la banana.

El objetivo consiste en lograr que el mono obtenga una banana utilizando distintas acciones válidas dentro del entorno.

El sistema representa:

- Posición del mono.
- Posición de la caja.
- Estado del mono.
- Si tiene o no la banana.

Las acciones implementadas fueron:

- Caminar.
- Empujar la caja.
- Subirse a la caja.
- Tomar la banana.

Ejemplo de consulta:

```
resolver(escenario(puerta, piso, ventana, sin_banana), X).
```

La consulta devuelve la secuencia de pasos necesarios para resolver el problema.

Torres de Hanoi

También se implementó el problema clásico de las Torres de Hanoi utilizando recursividad.

El programa mueve discos entre torres siguiendo las reglas del problema:

1. Solo puede moverse un disco a la vez.
2. Un disco grande no puede colocarse sobre uno pequeño.
3. Se utiliza una torre auxiliar para completar la solución.

Ejemplo de ejecución:

```
iniciar_hanoi(3).
```

El sistema muestra automáticamente los movimientos necesarios para resolver el problema.

Ejemplo básico en Prolog

```
saludo :-  
    write('Hola Mundo desde Prolog').
```

Conclusión

Durante esta práctica se comprendieron los fundamentos del paradigma lógico mediante el lenguaje Prolog.

Se observó que este paradigma se diferencia considerablemente de la programación imperativa y funcional, ya que el programador se enfoca en definir relaciones y conocimiento en lugar de especificar instrucciones secuenciales detalladas.

Además, se aprendió el uso de hechos, reglas, consultas y recursividad para resolver problemas clásicos mediante inferencia lógica automática. También se comprendió la importancia de mecanismos como la unificación y el backtracking dentro del funcionamiento interno de Prolog.